

Week 12

Audio



THE UNIVERSITY OF
SYDNEY



Student Survey is Out!

- Please take some time to evaluate myself, the lecture content, the tutorial content, and your tutors!!!
- If you think we did a bad job, tell us!
- If we did a good job, still tell us!
- Do you have comments? Tell us!
- We want to know all the good and the bad to improve the content and ourselves!
- Remember that the process is anonymous, so we won't know who wrote what.
- We would appreciate it greatly if you take some time to evaluate us.

Next Week Lecture

- We will have some pizza ~~and beer~~.
- Please come over to chat with me, your peers, about your final project, about MIDEA, about life, about anything you want.
- Pizza will not be unlimited.
- Do you guys even like pizza?

Next Week Tutorial

- Tutorials will be working out with your tutors any final issue you have with the assessment.
 - On Monday, we will provide a booking sheet for each student group to book a time slot with the tutor.
 - The time slot will be during the tutorial time.
 - Attendance will be taken during this one-to-one consultation time.
 - Please let us know where you are in the project and if you have any issues or questions.
 - The remaining time, you are encouraged to stay in the tutorial, work on your project, and complete the student survey 😊.
 - The consultation time will be calculated based on the number of students in the tutorial.

This week (Final Lecture of the Semester)

- Sound in P5.js

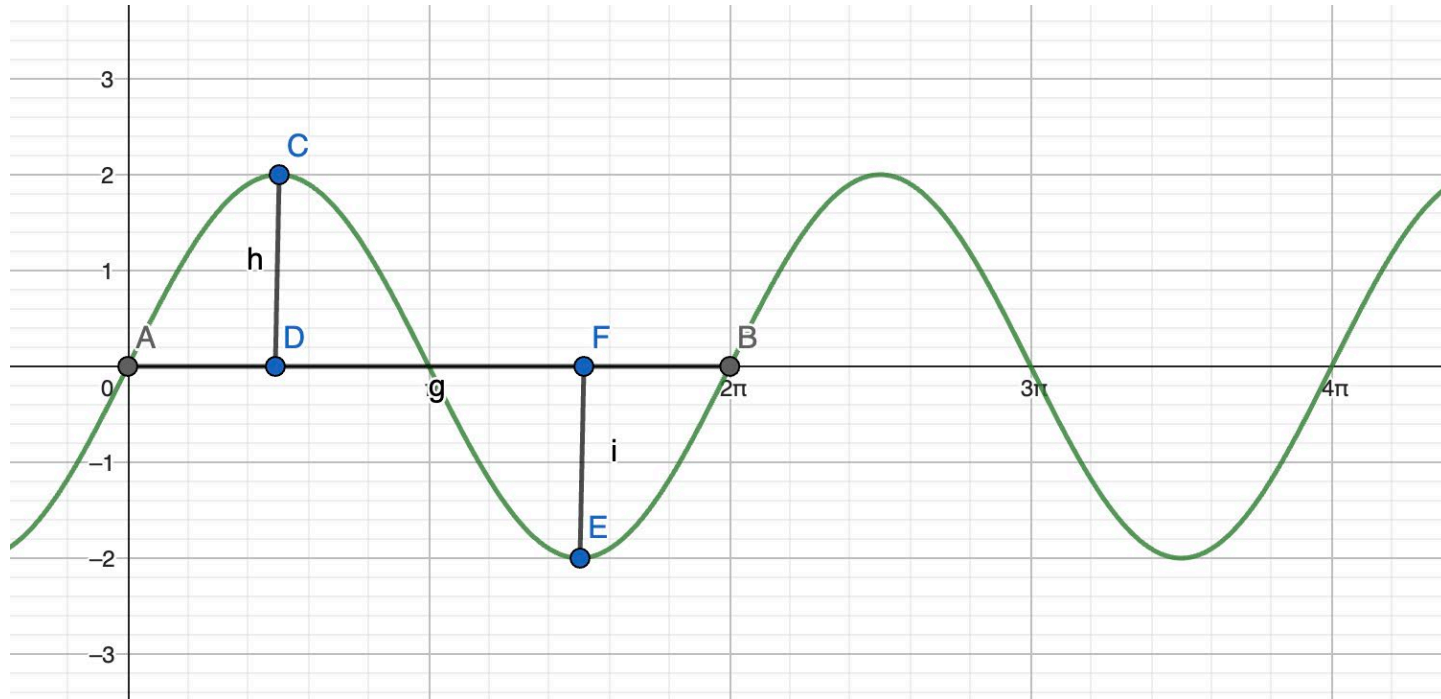
Basics of Sound



Basics of Sound

- To understand sound, you need to understand waves.
- Last week, we had a look at overlapping waves to create randomness.
- This week, we will look at the characteristics of waves so you can understand more about how sound behaves.

Basics of Sound: Wave Terminology



Basics of Sound: Wavelength

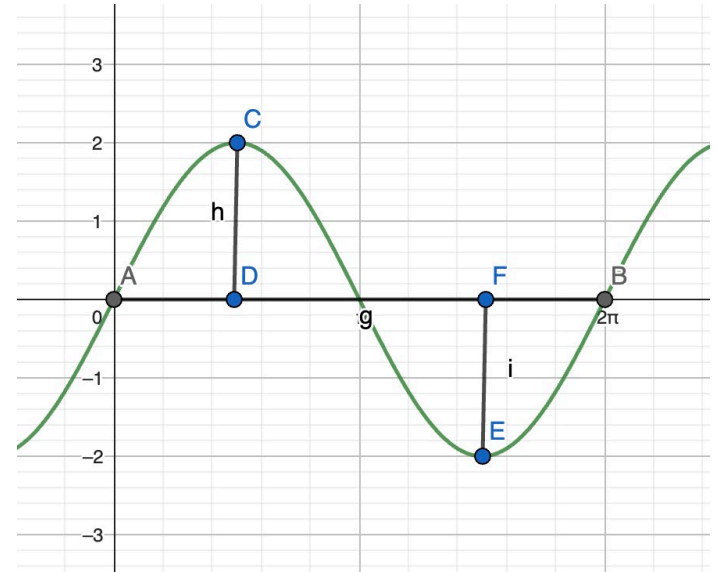
Wavelength (Points A and B)

The length of the single oscillation or a single cycle.

It goes from 0 to 2π .

It is like the space between repeating patterns.

The wavelength is the length of one pattern before the following pattern starts.



Basics of Sound: Frequency

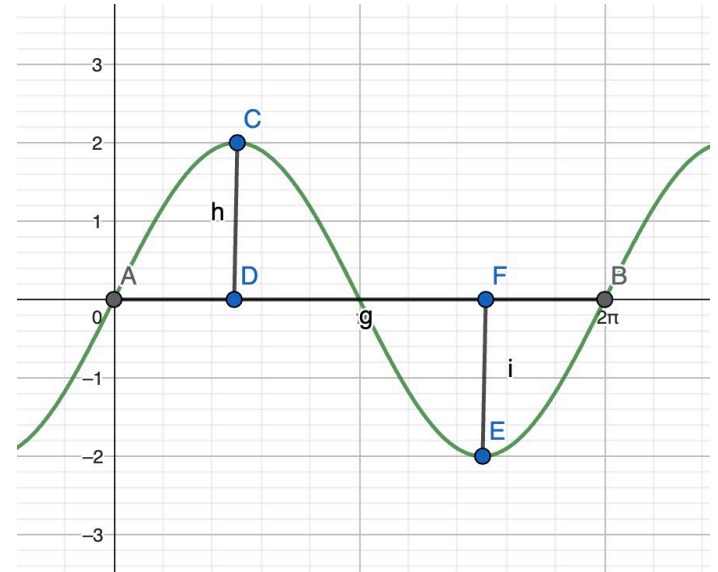
Frequency

The number of wavelengths in one (1) second.

Measured in Hertz (Hz).

In sound, it controls the pitch. Higher frequency means high-pitch sound.

Think of frequency as how many times or how often a pattern repeats itself.



Basics of Sound: Amplitude

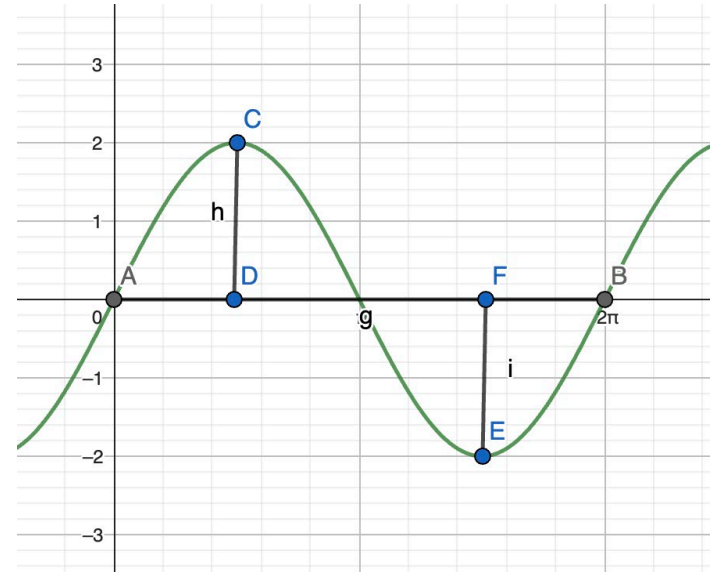
Amplitude (Points CD/EF)

The “height” of the wave.

In $y = 2 \sin x$ it is the number 2.

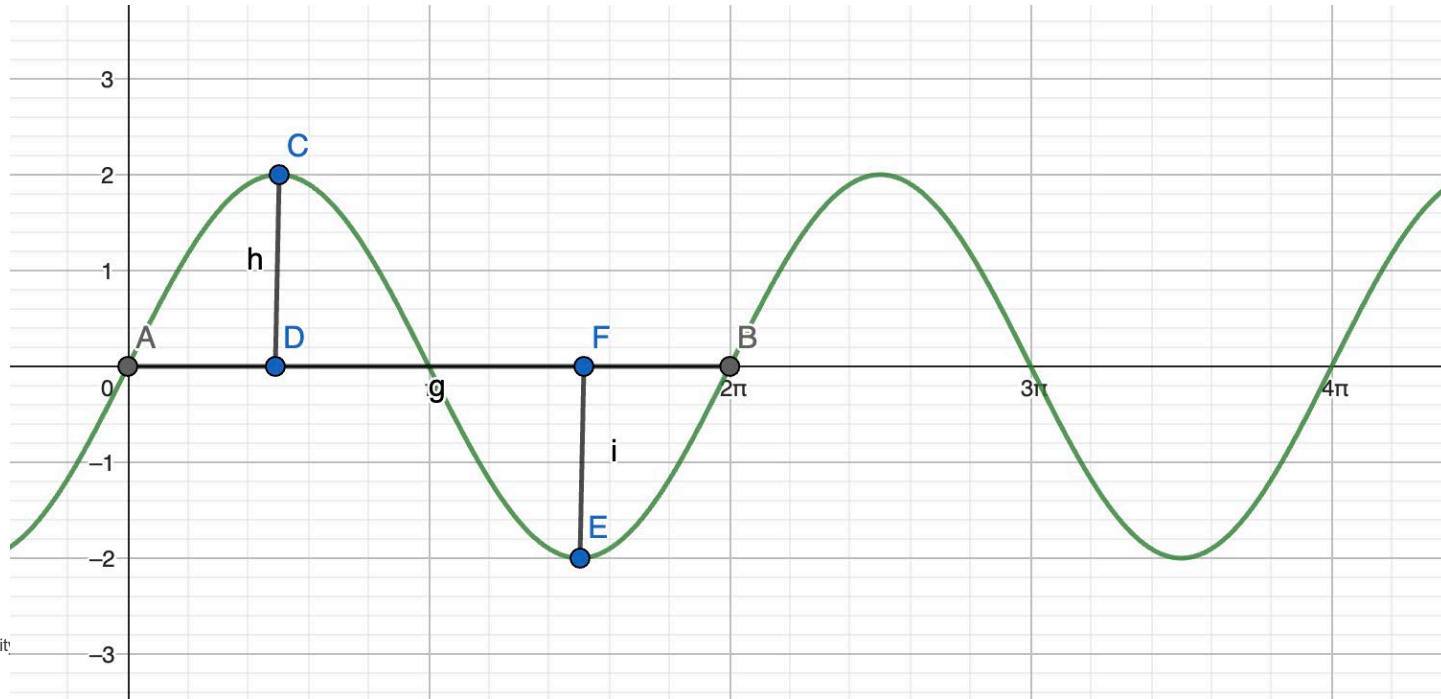
It controls **loudness** and **intensity**.

Higher amplitude, louder sound.



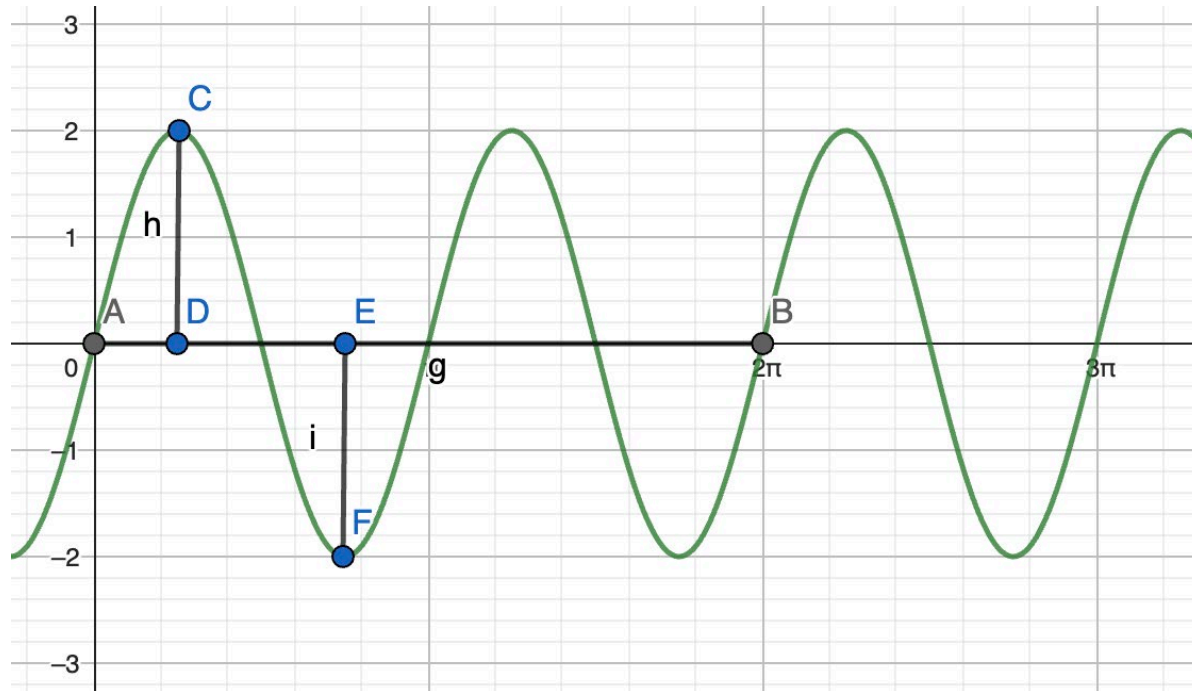
Basics of Sound

- As mentioned, this is the function to produce the signal in the example.
- $y = 2 \sin x$



Basics of Sound

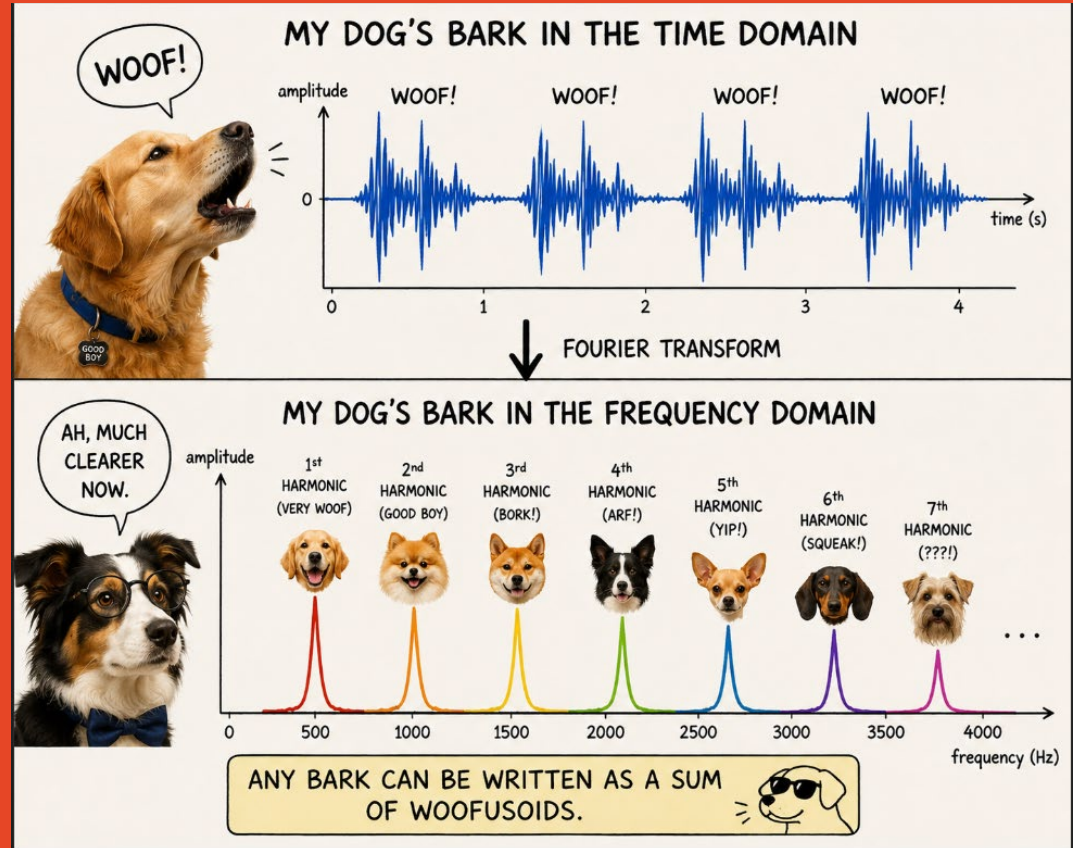
- Note how if we now modify it to $y = 2 \sin 2x$, we get half the wavelength



Questions?



Fourier Transform

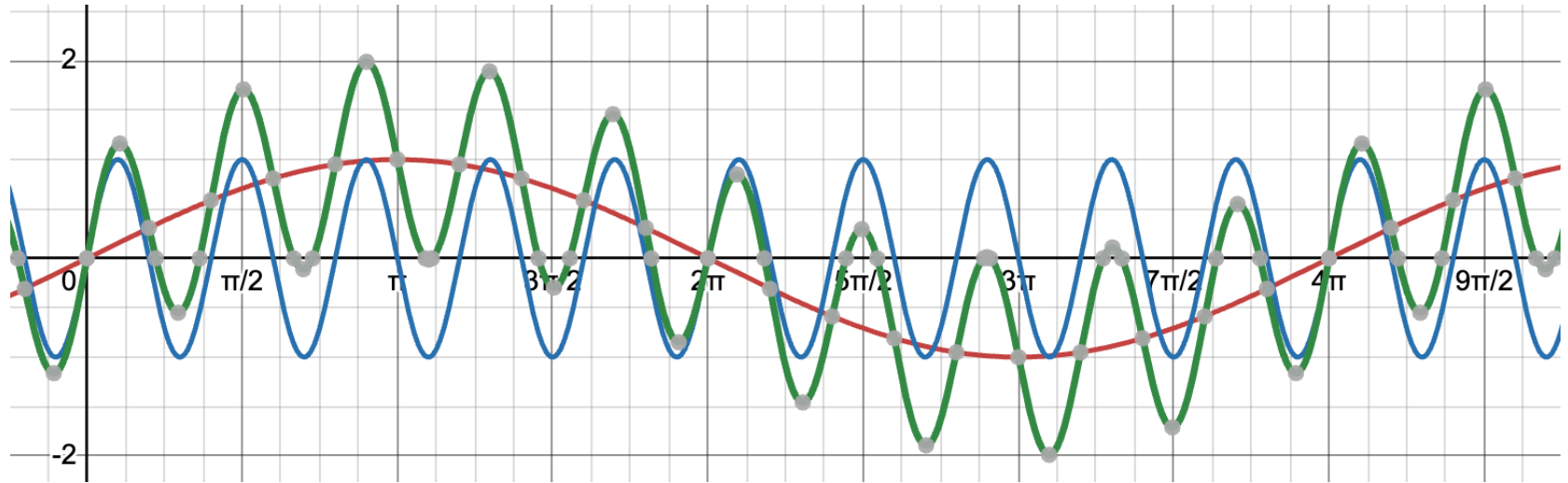


Fast Fourier Transform

- Fourier transform is a helpful tool for processing audio (more about this in a moment).
- How does it help?

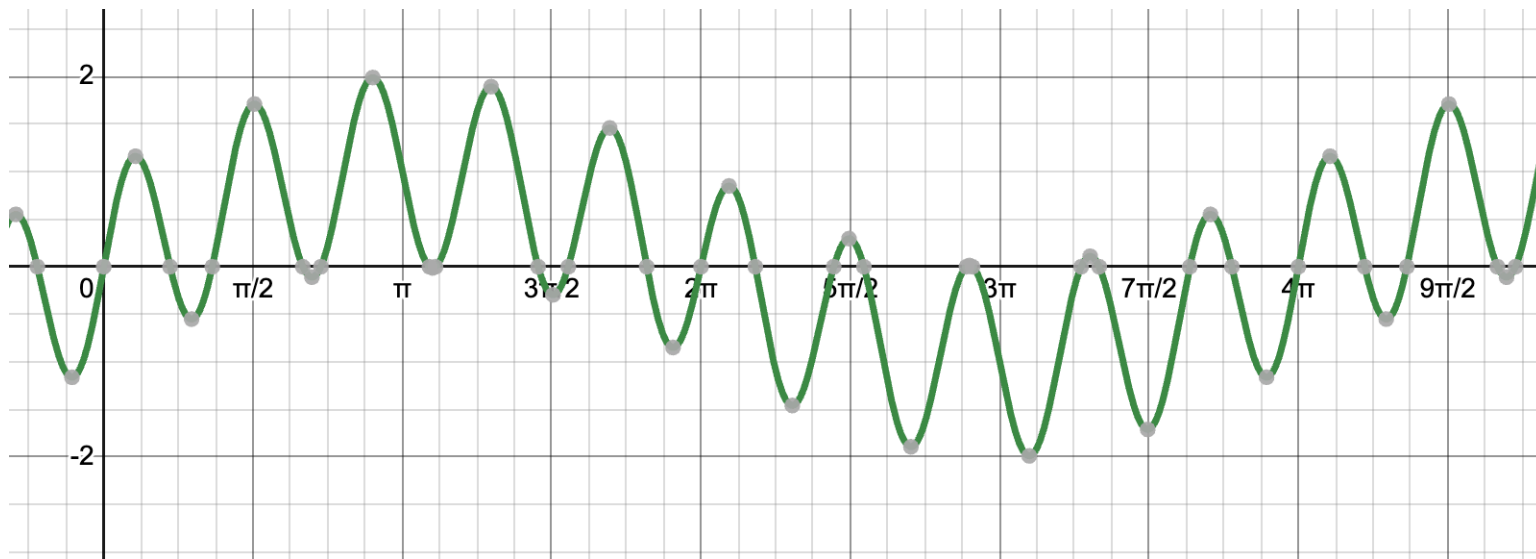
Fast Fourier Transform

- Remember Week 11.
- Remember how we discussed overlapping signals to create patterns?



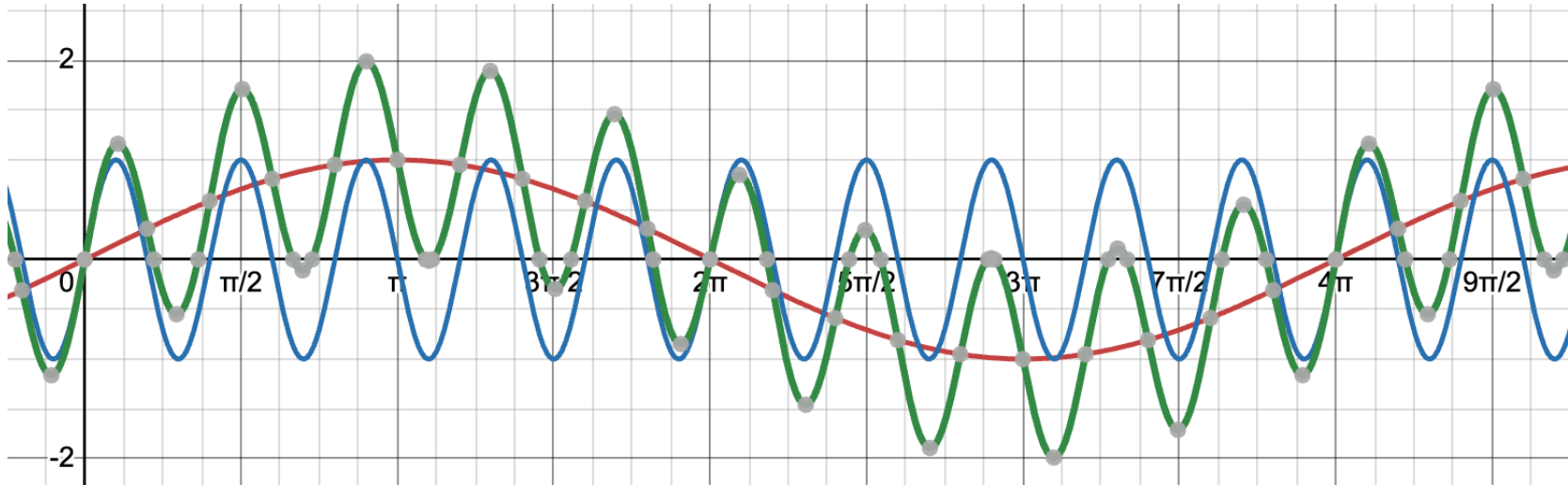
Fast Fourier Transform

- Fast Fourier Transform (or FFT) does the opposite of what we did last week.
- It uses this as an input.



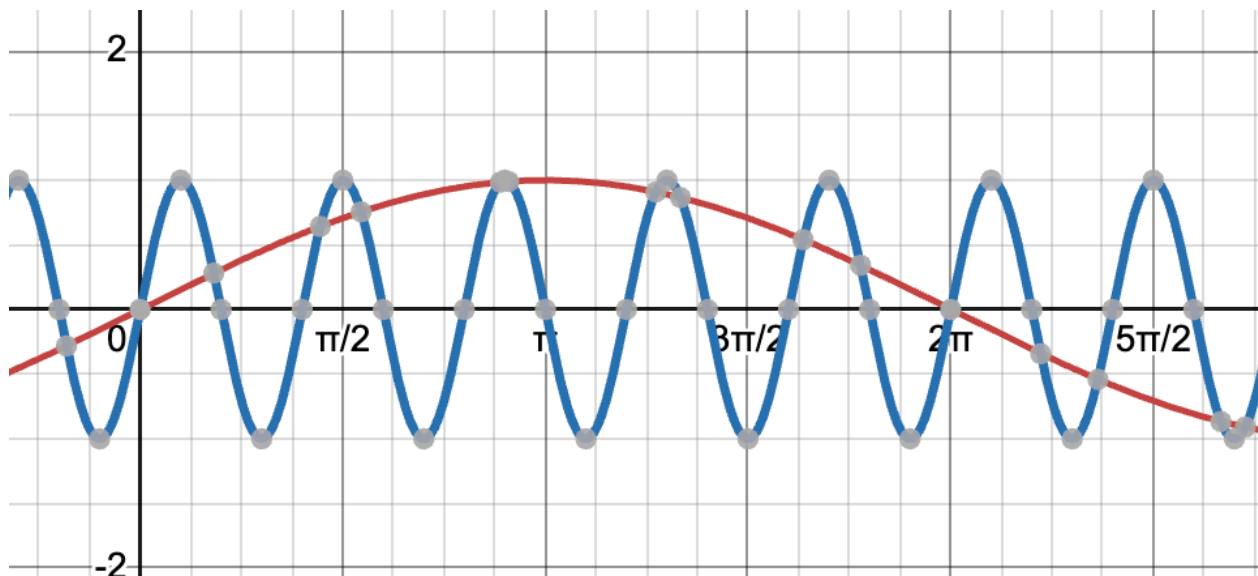
Fast Fourier Transform

- Fast Fourier Transform (or FFT) does the opposite of what we did last week.
- Uses math to identify these



Fast Fourier Transform

- Fast Fourier Transform (or FFT) does the opposite of what we did last week.
- And this is the output (sort of).



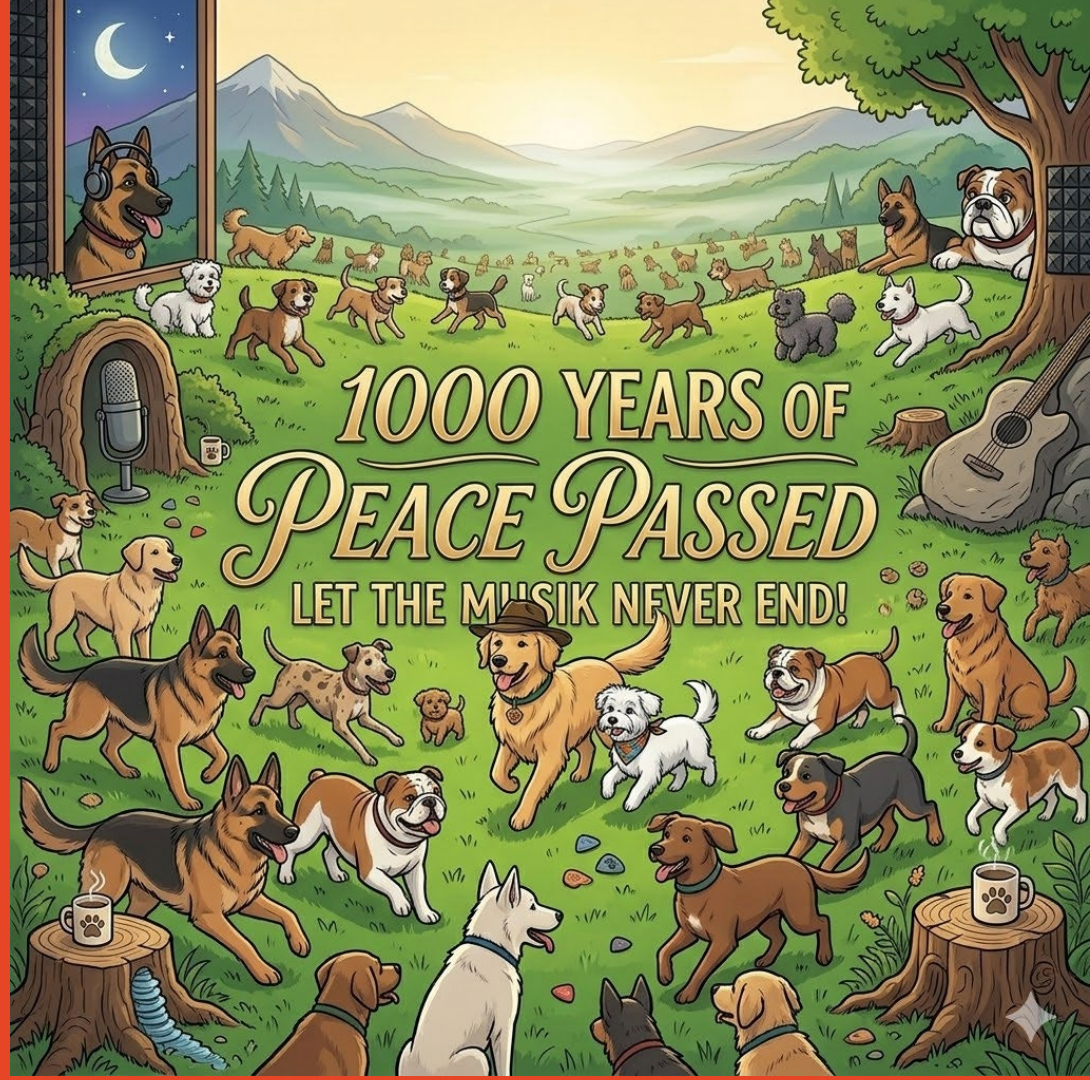
Fast Fourier Transform

- There are many uses for FFT beyond sound design.
- If you want to learn the technicalities of FFT and the process behind that, have a look at this video (also in Canvas).
- <https://www.youtube.com/watch?v=spUNpyF58BY>
- We will discuss FFT and its applications in sound later in the lecture and during the tutorials.

Questions?



p5.sound



Load an audio file and play it

- This is the code for loading and playing an audio file. Let's go through it!
- Note that the code in the Canvas lecture content **MIGHT** not work.
- Some browsers require the user to interact before playing music (ie a play and stop button) for security reasons.
- This code is guaranteed to work in any browser.

```
1  let audio;
2  let slider;
3
4  let playing = false;
5
6  function preload() {
7    audio = loadSound('batch.mp3');
8  }
9
10 function setup() {
11   createCanvas(400, 400);
12   //audio.play();
13   //audio.loop();
14   //audio.stop();
15   //audio.setVolume(0.5); //0 (no volume) to 1 (full volume)
16   //createSlider(min, max, value, step)
17   slider = createSlider(0, 1, 0.5, 0.02);
18 }
19
20 function draw() {
21   background(220);
22   audio.setVolume(slider.value());
23 }
24
25 function togglePlay(){
26   if (!playing) {
27     audio.play();
28     playing = true;
29     console.log("playing!")
30   } else {
31     audio.pause();
32     playing = false;
33     console.log("pausing!")
34   }
35 }
36
37 function keyPressed(){
38   if (keyCode == 32){
39     togglePlay();
40   }
41 }
```

Load and audio file and play it

```
6  function preload() {  
7      audio = loadSound('batch.mp3');  
8  }
```

- As we learned when loading images, all heavy assets must be loaded in the *preload* function.
- Use the *loadSound* function to load a sound file to your sketch.

Load an audio file and play it

```
10  function setup() {  
11      createCanvas(400, 400);  
12      //audio.play();  
13      //audio.loop();  
14      //audio.stop();  
15      //audio.setVolume(0.5); //0 (no volume) to 1 (full volume)  
16      //createSlider(min, max, value, step)  
17      slider = createSlider(0, 1, 0.5, 0.02);  
18  }
```

Load an audio file and play it

```
10  function setup() {  
11      createCanvas(400, 400);  
12      //audio.play();  
13      //audio.loop();  
14      //audio.stop();  
15      //audio.setVolume(0.5); //0 (no volume) to 1 (full volume)  
16      //createSlider(min, max, value, step)  
17      slider = createSlider(0, 1, 0.5, 0.02);  
18  }
```

All the options to adjust the audio of the sketch.

Load an audio file and play it

```
10  function setup() {  
11      createCanvas(400, 400);  
12      //audio.play();  
13      //audio.loop();  
14      //audio.stop();  
15      //audio.setVolume(0.5); //0 (no volume) to 1 (full volume)  
16      //createSlider(min, max, value, step)  
17      slider = createSlider(0, 1, 0.5, 0.02);  
18  }
```

The inputs are min and max values,
The default value will be 0.5
Every change in the slider will increase 0.02

Load an audio file and play it

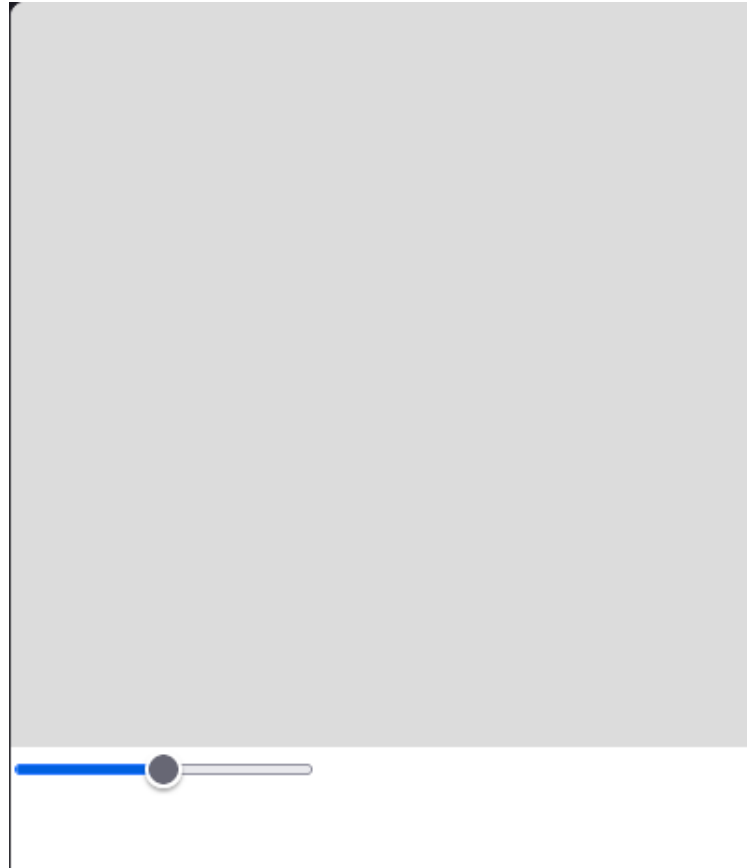
```
20  function draw() {  
21      background(220);  
22      audio.setVolume(slider.value());  
23  }
```

Load an audio file and play it

```
20  function draw() {  
21      background(220);  
22      audio.setVolume(slider.value());  
23  }
```

Set the volume of your audio file.
Use a number between 0 and 1,
using the slider as the volume
setter.

DEMO!



Playing audio in multiple speakers

- P5.sound also allow us to manage the left or right speaker.
- The code is almost the same but with some differences.

```
10  function setup() {  
11      createCanvas(400, 400);  
12      // audio.play();  
13      //audio.loop();  
14      //audio.stop();  
15      //audio.setVolume(0.5); //0 (no volume) to 1 (full volume)  
16      slider = createSlider(0, 1, 0.5, 0.02);  
17      sliderPan = createSlider(-1, 1, 0, 0.01);  
18      playing = false;  
19  }
```

Playing audio in multiple speakers

- P5.sound also allow us to manage the left or right speaker.
- The code is almost the same but with some differences.

We have two sliders, one for volume, and one that pans between speakers

```
10  function setup() {  
11      createCanvas(400, 400);  
12      // audio.play();  
13      //audio.loop();  
14      //audio.stop();  
15      //audio.setVolume(0.5); //0 (no volume) to 1 (full volume)  
16      slider = createSlider(0, 1, 0.5, 0.02);  
17      sliderPan = createSlider(-1, 1, 0, 0.01);  
18      playing = false;  
19  }
```

Playing audio in multiple speakers

```
function draw() {  
  background(220);  
  audio.setVolume(slider.value());  
  audio.pan(sliderPan.value());  
}
```

Playing audio in multiple speakers

audio.pan will
shift the sound
between
speakers

```
function draw() {  
  background(220);  
  audio.setVolume(slider.value());  
  audio.pan(sliderPan.value());  
}
```

Questions?

Animating with Sound



Visualising amplitude

- This is an example of how to animate the size of a circle based on an audio piece.
- Let's have a look.

```
1  let audio;
2  let button;
3  let amp;
4
5  function preload() {
6    audio = loadSound('batch.mp3');
7  }
8
9  function setup() {
10     createCanvas(400, 400);
11     button = createButton('PLAY');
12     button.mousePressed(togglePlaying);
13     amp = new p5.Amplitude();
14   }
15
16   function draw() {
17     background(220);
18     print(amp.getLevel());
19     r = amp.getLevel() * width;
20     fill('red');
21     circle(200, 200, r);
22   }
23
24   function togglePlaying() {
25     if (!audio.isPlaying()) {
26       audio.play();
27       audio.setVolume(1.0);
28       button.html("PAUSE");
29     } else {
30       audio.stop();
31       button.html("PLAY");
32     }
33   }
```

Visualising Amplitude

```
function setup() {  
  createCanvas(400, 400);  
  button = createButton('PLAY');  
  button.mousePressed(togglePlaying);  
  amp = new p5.Amplitude();  
}
```

Visualising Amplitude

```
function setup() {  
  createCanvas(400, 400);  
  button = createButton('PLAY');  
  button.mousePressed(togglePlaying);  
  amp = new p5.Amplitude();  
}
```

We use p5's *createButton* function to create a button.
We also use the *mousePressed* function to give it functionality.

Visualising Amplitude

```
function setup() {  
  createCanvas(400, 400);  
  button = createButton('PLAY');  
  button.mousePressed(togglePlaying);  
  amp = new p5.Amplitude();  
}
```

The amplitude class is “listening” to your audio played within your browser and can “report” on its status.

Visualising Amplitude

```
16  function draw() {  
17      background(220);  
18      print(amp.getLevel());  
19      r = amp.getLevel() * width;  
20      fill('red');  
21      circle(200, 200, r);  
22  }
```

Visualising Amplitude

```
16  function draw() {  
17      background(220);  
18      print(amp.getLevel());  
19      r = amp.getLevel() * width;  
20      fill('red');  
21      circle(200, 200, r);  
22  }
```

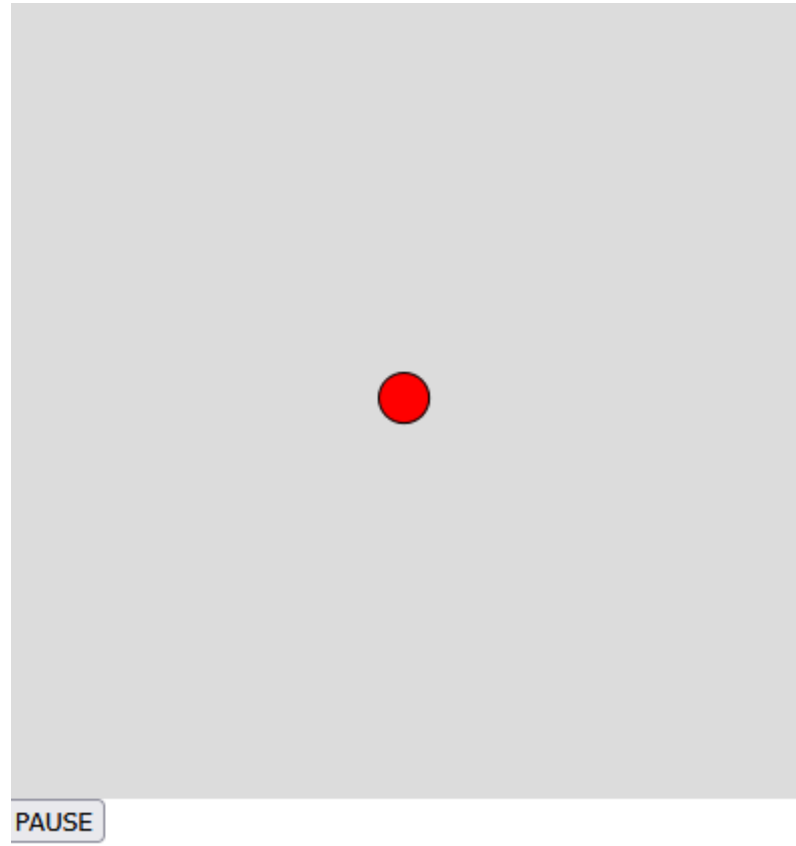
The amplitude returns a value between 1 and 0 based on the amplitude of the sound detected.

Visualising Amplitude

```
16  function draw() {  
17      background(220);  
18      print(amp.getLevel());  
19      r = amp.getLevel() * width;  
20      fill('red');  
21      circle(200, 200, r);  
22  }
```

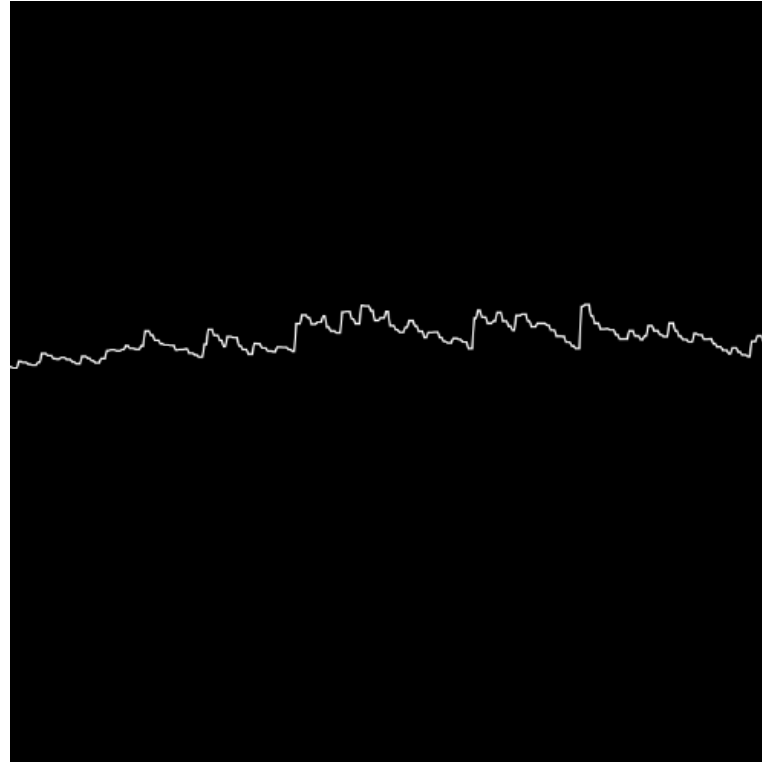
We will adjust the size of the ball based on the amplitude level and the width of the canvas. If the noise amplitude gets to the noisiest level (1), the circle will grow to occupy the full canvas.

Demo Time!



Visualising Amplitude v2

Demo First



PAUSE

Visualising Amplitude v2

```
17 function draw() {
18     background(0);
19     let r = amp.getLevel();
20     ampArray.push(r);
21     stroke(255);
22     noFill();
23     beginShape();
24     for (i = 0; i < ampArray.length; i++) {
25         y = map(ampArray[i], 0, 1, height / 2, 0);
26         //vertex(i,height/2);
27         vertex(i, y);
28     }
29     endShape();
30
31     if (ampArray.length > width) {
32         ampArray.splice(0, 1);
33     }
34 }
```


Visualising Amplitude v2

```
17 function draw() {  
18   background(0);  
19   let r = amp.getLevel();  
20   ampArray.push(r);  
21   stroke(255);  
22   noFill();  
23   beginShape();  
24   for (i = 0; i < ampArray.length; i++) {  
25     y = map(ampArray[i], 0, 1, height / 2, 0);  
26     //vertex(i,height/2);  
27     vertex(i, y);  
28   }  
29   endShape();  
30  
31   if (ampArray.length > width) {  
32     ampArray.splice(0, 1);  
33   }  
34 }
```

Get the current
amplitude value.
Nothing new.

Visualising Amplitude v2

```
17 function draw() {
18   background(0);
19   let r = amp.getLevel();
20   ampArray.push(r);
21   stroke(255),
22   noFill();
23   beginShape();
24   for (i = 0; i < ampArray.length; i++) {
25     y = map(ampArray[i], 0, 1, height / 2, 0);
26     //vertex(i,height/2);
27     vertex(i, y);
28   }
29   endShape();
30
31   if (ampArray.length > width) {
32     ampArray.splice(0, 1);
33   }
34 }
```

The line you see is just an array of changes to the amplitude.

In the ampArray we store all the values.

Visualising Amplitude v2

```
17 function draw() {
18   background(0);
19   let r = amp.getLevel();
20   ampArray.push(r);
21   stroke(255);
22   noFill();
23   beginShape();
24   for (i = 0; i < ampArray.length; i++) {
25     y = map(ampArray[i], 0, 1, height / 2, 0);
26     //vertex(i,height/2);
27     vertex(i, y);
28   }
29   endShape();
30
31   if (ampArray.length > width) {
32     ampArray.splice(0, 1);
33   }
34 }
```

NEW:

If you want to create a custom shape (or line) you use begin shape.

Visualising Amplitude v2

```
17 function draw() {  
18   background(0);  
19   let r = amp.getLevel();  
20   ampArray.push(r);  
21   stroke(255);  
22   noFill();  
23   beginShape();  
24   for (i = 0; i < ampArray.length; i++) {  
25     y = map(ampArray[i], 0, 1, height / 2, 0);  
26     //vertex(i, height/2);  
27     vertex(i, y);  
28   }  
29   endShape();  
30  
31   if (ampArray.length > width) {  
32     ampArray.splice(0, 1);  
33   }  
34 }
```

NEW:

Then, you give the location of all the points that make up the shape you want to draw.

Visualising Amplitude v2

```
17 function draw() {  
18   background(0);  
19   let r = amp.getLevel();  
20   ampArray.push(r);  
21   stroke(255);  
22   noFill();  
23   beginShape();  
24   for (i = 0; i < ampArray.length; i++) {  
25     y = map(ampArray[i], 0, 1, height / 2, 0);  
26     //vertex(i,height/2);  
27     vertex(i, y);  
28   }  
29   endShape();  
30  
31   if (ampArray.length > width) {  
32     ampArray.splice(0, 1);  
33   }  
34 }
```

NEW:

Then you use
endShape to tell
p5 you finish you
complex shape.

Visualising Amplitude v2

```
17 function draw() {  
18   background(0);  
19   let r = amp.getLevel();  
20   ampArray.push(r);  
21   stroke(255);  
22   noFill();  
23   beginShape();  
24   for (i = 0; i < ampArray.length; i++) {  
25     y = map(ampArray[i], 0, 1, height / 2, 0);  
26     //vertex(i,height/2);  
27     vertex(i, y);  
28   }  
29   endShape();  
30  
31   if (ampArray.length > width) {  
32     ampArray.splice(0, 1);  
33   }  
34 }
```

To calculate the line's height, we map the amp value from 0 -1 to (height/2)—0.

Does anyone want to guess why this is?

Visualising Amplitude v2

```
17 function draw() {  
18   background(0);  
19   let r = amp.getLevel();  
20   ampArray.push(r);  
21   stroke(255);  
22   noFill();  
23   beginShape();  
24   for (i = 0; i < ampArray.length; i++) {  
25     y = map(ampArray[i], 0, 1, height / 2, 0);  
26     //vertex(i,height/2);  
27     vertex(i, y);  
28   }  
29   endShape();  
30  
31   if (ampArray.length > width) {  
32     ampArray.splice(0, 1);  
33   }  
34 }
```

To calculate the line's height, we map the amp value from 0 -1 to (height/2)—0.

So that the amplitude of 0 starts at the middle of the canvas, and the amplitude of 1 goes to the top of the canvas.

Visualising Amplitude v2

```
17 function draw() {  
18   background(0);  
19   let r = amp.getLevel();  
20   ampArray.push(r);  
21   stroke(255);  
22   noFill();  
23   beginShape();  
24   for (i = 0; i < ampArray.length; i++) {  
25     y = map(ampArray[i], 0, 1, height / 2, 0);  
26     //vertex(i,height/2);  
27     vertex(i, y);  
28   }  
29   endShape();  
30  
31   if (ampArray.length > width) {  
32     ampArray.splice(0, 1);  
33   }  
34 }
```

NEW:

The splice function removes x elements at the index i.

If I call: *splice* (5, 7);
It means removing 7 elements starting at index 5.

So, what does this splice function do?

Visualising Amplitude v2

```
17 function draw() {  
18   background(0);  
19   let r = amp.getLevel();  
20   ampArray.push(r);  
21   stroke(255);  
22   noFill();  
23   beginShape();  
24   for (i = 0; i < ampArray.length; i++) {  
25     y = map(ampArray[i], 0, 1, height / 2, 0);  
26     //vertex(i,height/2);  
27     vertex(i, y);  
28   }  
29   endShape();  
30  
31   if (ampArray.length > width) {  
32     ampArray.splice(0, 1);  
33   }  
34 }
```

NEW:

The splice function removes x elements at the index i.

If I call: *splice (5, 7);*
It means removing 7 elements starting at index 5.

Removes 1 element at index 0.
It makes it look like the line is progressing.

Questions?

Using FFT in p5



Visualising the waveforms through FFT

```
1  let audio;
2  let button;
3  let fft;
4
5  function preload() {
6    audio = loadSound('batch.mp3');
7  }
8
9  function setup() {
10     createCanvas(512, 512);
11     button = createButton('PLAY');
12     button.mousePressed(togglePlaying);
13     fft = new p5.FFT(0, 256); //0 refers to smoothing, 256 refers to the number of frequency bins.
14 }
```

We create a FFT object.

Visualising the waveforms through FFT

```
1  let audio;
2  let button;
3  let fft;
4
5  function preload() {
6    audio = loadSound('batch.mp3');
7  }
8
9  function setup() {
10    createCanvas(512, 512);
11    button = createButton('PLAY');
12    button.mousePressed(togglePlaying);
13    fft = new p5.FFT(0, 256); //0 refers to smoothing, 256 refers to the number of frequency bins.
14  }
```

The first parameter refers to the smoothing of the signal.

Visualising the waveforms through FFT

```
1  let audio;
2  let button;
3  let fft;
4
5  function preload() {
6    audio = loadSound('batch.mp3');
7  }
8
9  function setup() {
10    createCanvas(512, 512);
11    button = createButton('PLAY');
12    button.mousePressed(togglePlaying);
13    fft = new p5.FFT(0, 256); //0 refers to smoothing, 256 refers to the number of frequency bins.
14  }
```

The second parameter means you will divide the signal into 256 sub-signals.

Visualising the waveforms through FFT

```
1  let audio;
2  let button;
3  let fft;
4
5  function preload() {
6    audio = loadSound('batch.mp3');
7  }
8
9  function setup() {
10     createCanvas(512, 512);
11     button = createButton('PLAY');
12     button.mousePressed(togglePlaying);
13     fft = new p5.FFT(0, 256); //0 refers to smoothing, 256 refers to the number of frequency bins.
14 }
```

Think of 256 as the resolution of the audio.

Visualising waveforms through FFT

```
16  function draw() {
17      background(0);
18      let ampArray = fft.analyze();
19      stroke(255);
20      noFill();
21      console.log(ampArray.length);
22      for (i = 0; i < ampArray.length; i++) {
23          y = map(ampArray[i], 0, 255, height, 0);
24          strokeWeight(2);
25          fill(y, y * 0.5, y * 0.1);
26          //line(i,height,i,y);
27          //line(i+100, height, i+100, y);
28          rect(i * 10, y, 10, height - y);
29      }
30  }
```


Visualising waveforms through FFT

```
16 function draw() {  
17   background(0);  
18   let ampArray = fft.analyze();  
19   stroke(255);  
20   noFill();  
21   console.log(ampArray.length);  
22   for (i = 0; i < ampArray.length; i++) {  
23     y = map(ampArray[i], 0, 255, height, 0);  
24     strokeWeight(2);  
25     fill(y, y * 0.5, y * 0.1);  
26     //line(i,height,i,y);  
27     //line(i+100, height, i+100, y);  
28     rect(i * 10, y, 10, height - y);  
29   }  
30 }
```

The *analyze()* function will return an array of size 256 (a parameter we set when creating the FFT object).

Each index of the array will contain the amplitude of each sub-signal.

Visualising waveforms through FFT

```
16  function draw() {  
17      background(0);  
18      let ampArray = fft.analyze();  
19      stroke(255);  
20      noFill();  
21      console.log(ampArray.length);  
22      for (i = 0; i < ampArray.length; i++) {  
23          y = map(ampArray[i], 0, 255, height, 0);  
24          strokeWeight(2);  
25          fill(y, y * 0.5, y * 0.1);  
26          //line(i,height,i,y);  
27          //line(i+100, height, i+100, y);  
28          rect(i * 10, y, 10, height - y);  
29      }  
30  }
```

The value returned will be between 0 and 255, so we need to map it to be between height (the bottom of the screen) and 0 (the top of the screen).

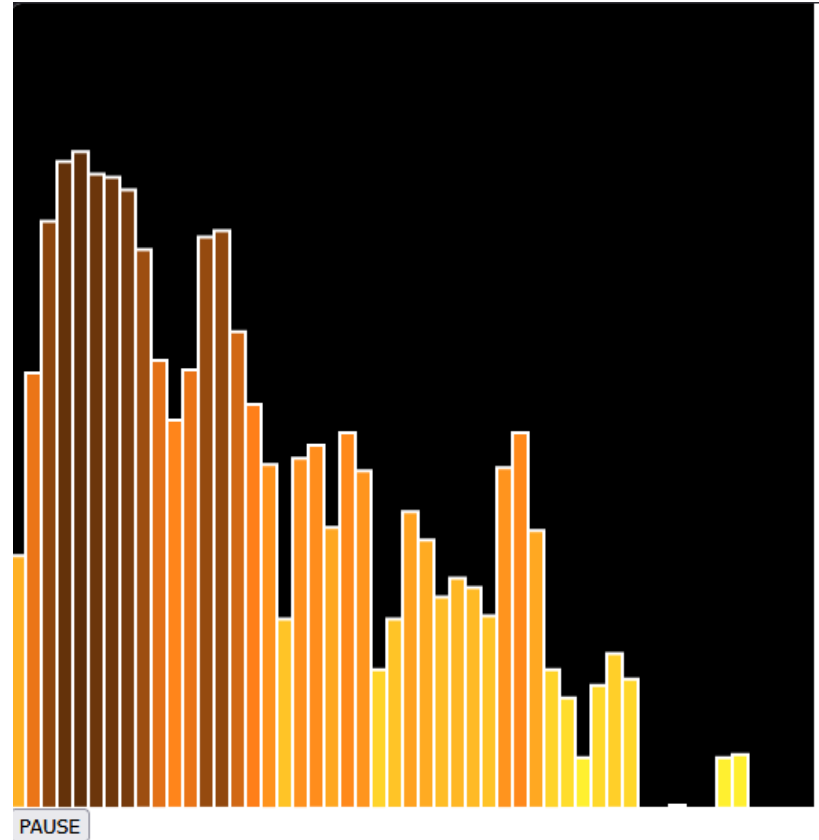
Visualising waveforms through FFT

```
16  function draw() {  
17      background(0);  
18      let ampArray = fft.analyze();  
19      stroke(255);  
20      noFill();  
21      console.log(ampArray.length);  
22      for (i = 0; i < ampArray.length; i++) {  
23          y = map(ampArray[i], 0, 255, height, 0);  
24          strokeWeight(2);  
25          fill(y, y * 0.5, y * 0.1);  
26          //line(i,height,i,y);  
27          //line(i+100, height, i+100, y);  
28          rect(i * 10, y, 10, height - y);  
29      }  
30  }
```

This creates a yellowish colour for low amplitude, orange-ish for medium, and black for high amplitude!

Visualising the waveforms through FFT

Play the Demo



Closing Thoughts!

Try loading different songs to see how they create different waves.

The audio library for p5 is a cool way to add transformations to your pieces.

Try mapping amplitude to sizes, colours, or positions to create cool pieces!

Questions?